

# LINGI2364: Frequent Itemset Mining

Yassine Zeamari - group: 53

March 2026

## 1 Introduction

Frequent itemset mining is an important problem in data mining. The goal is to find groups of items that often appear together in a transactional dataset.

In this project, we implemented and compared three algorithms: Apriori, Eclat, and FP-Growth. The objective is to study how the different algorithmic strategies affect performance, we evaluate the algorithms using several datasets and different minimum support thresholds. we analyse execution time, memory usage, and scalability.

## 2 Algorithm

### 2.1 Apriori

The algorithm works level by level. First it finds frequent items. Then it generates candidate itemsets of size two, then three, and so on. At each step the database must be scanned to compute the support of the candidates.

Although Apriori is simple to understand, it can become slow because it generates a large number of candidate itemsets.

For Apriori, we added a few optimizations compared to the basic pseudo-code from the course.

First, we used a hash-based pruning technique for generating candidate 2-itemsets. Instead of considering all possible pairs from  $L_1$ , we hashed the pairs observed in the transactions into buckets and only kept the pairs whose bucket count could reach the minimum support threshold. This reduced the size of  $C_2$ .

Second, after computing  $L_1$ , we filtered each transaction by removing all infrequent items.

### 2.2 Eclat

Instead of scanning the database many times, it transforms the dataset into a vertical format.

Each item is associated with the list of transaction identifiers in which it appears. The support of a larger itemset is computed by intersecting these transaction identifier sets.

### 2.3 FP-Growth

The FP-Growth algorithm works in two main steps. The dataset is first scanned to count the support of each item and remove infrequent ones.

Transactions are then inserted into the FP-tree after sorting their items by decreasing support. This ordering improves the compression of the tree because common prefixes are shared between transactions.

A header table is also maintained in order to quickly access all nodes corresponding to the same item. Frequent patterns are mined recursively by constructing conditional pattern bases and conditional FP-trees

### 3 Results

#### 3.1 Retail dataset

| MinSup | Apriori | Eclat | FP-Growth |
|--------|---------|-------|-----------|
| 0.9    | 45.15   | 86.46 | 45.15     |
| 0.8    | 45.15   | 86.46 | 45.15     |
| 0.7    | 45.15   | 86.46 | 45.15     |
| 0.6    | 45.15   | 86.46 | 45.15     |
| 0.5    | 48.57   | 86.46 | 45.15     |
| 0.4    | 49.50   | 88.96 | 45.15     |
| 0.3    | 49.50   | 88.96 | 45.15     |
| 0.2    | 49.51   | 88.96 | 45.15     |
| 0.1    | 50.23   | 90.58 | 45.15     |

Table 1: Peak memory usage (MB) on retail.dat

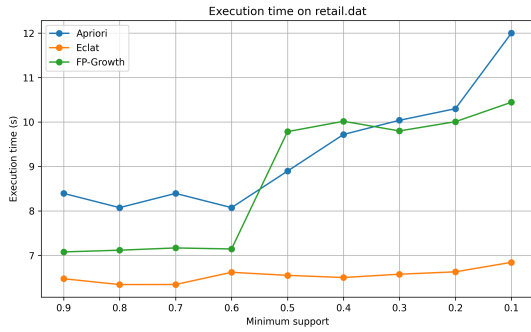


Figure 1: Execution time of the Apriori, Eclat, FP Growth algorithm

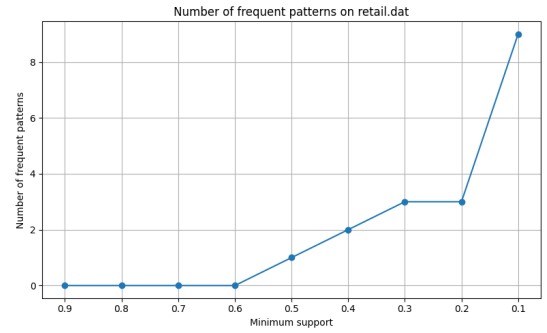


Figure 2: Number of frequent patterns

#### 3.2 chess dataset

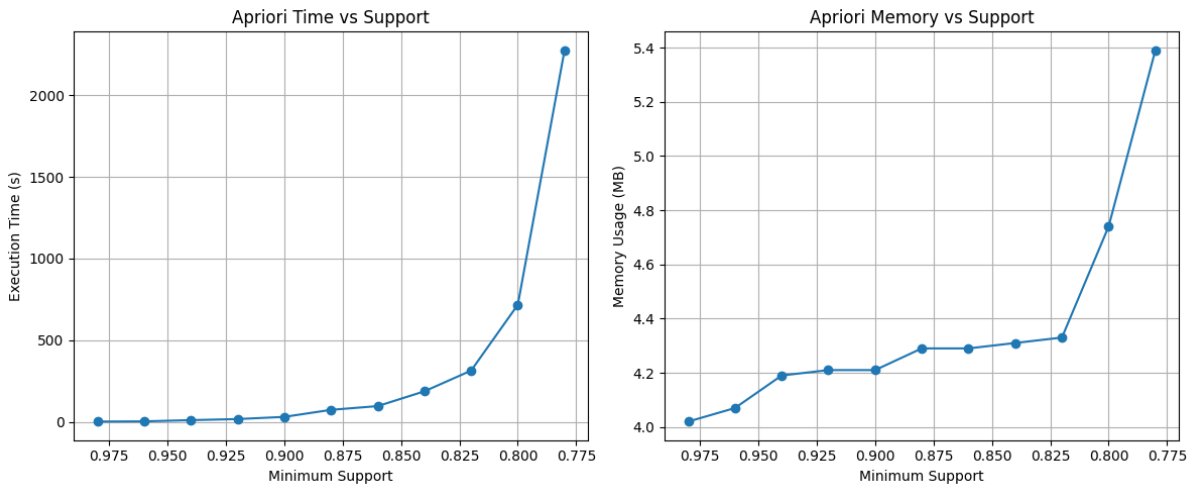


Figure 3: Execution time and memory usage of the Apriori algorithm

| MinSup | ECLAT     |             | FP-Growth |             |
|--------|-----------|-------------|-----------|-------------|
|        | Time (s)  | Memory (MB) | Time (s)  | Memory (MB) |
| 0.9    | 0.9980    | 15.68       | 1.5938    | 2.50        |
| 0.8    | 4.9109    | 24.82       | 4.1108    | 4.48        |
| 0.7    | 22.4600   | 37.70       | 20.3200   | 19.76       |
| 0.6    | 99.2639   | 78.17       | 77.2318   | 80.80       |
| 0.5    | 571.8755  | 345.28      | 290.7852  | 354.19      |
| 0.4    | 3315.6225 | 1882.08     | 1491.6210 | 1910.65     |

Table 2: Comparison of ECLAT and FP-Growth on the chess dataset

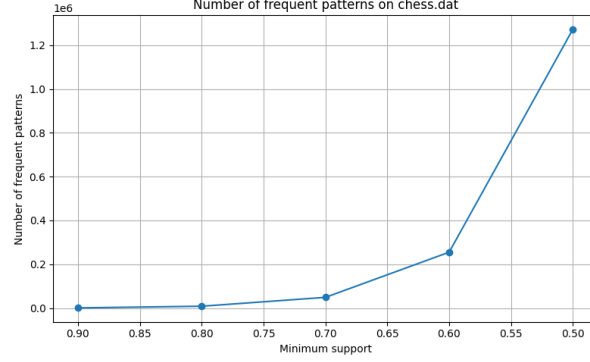


Figure 4: Number of frequent patterns

### 3.3 accidents dataset

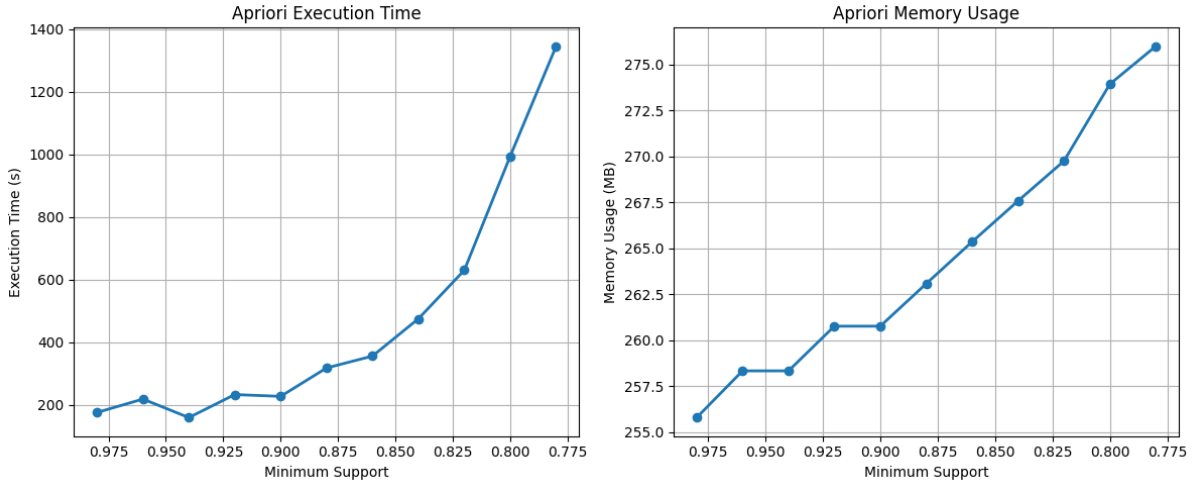


Figure 5: Execution time and memory usage of the Apriori algorithm

| MinSup | ECLAT     |             | FP-Growth |             |
|--------|-----------|-------------|-----------|-------------|
|        | Time (s)  | Memory (MB) | Time (s)  | Memory (MB) |
| 0.9    | 68.8717   | 782.23      | 125.5266  | 230.76      |
| 0.8    | 74.5871   | 970.24      | 164.0124  | 231.06      |
| 0.7    | 106.5313  | 1282.28     | 147.8698  | 232.80      |
| 0.6    | 170.0511  | 1762.39     | 180.1197  | 247.79      |
| 0.5    | 479.9387  | 2308.29     | 446.8456  | 283.41      |
| 0.4    | 1248.1719 | 2816.57     | 2408.0359 | 395.24      |

Table 3: Performance comparison of ECLAT and FP-Growth on the accidents dataset

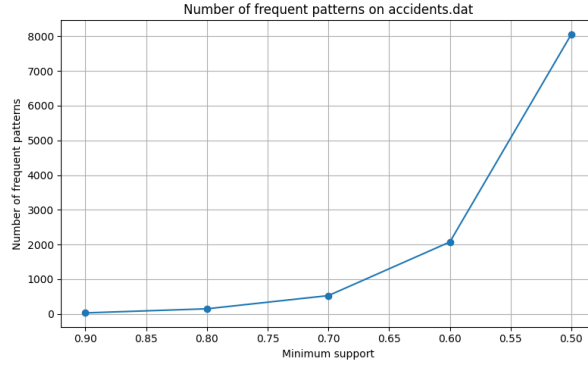


Figure 6: Number of frequent patterns

## 4 discussion

- for The retail dataset the transactions contain relatively few items. Because of this, the number of frequent patterns remains small even when the support decreases.

As a result, the three algorithms have similar execution times on this dataset. Eclat is slightly faster, but the difference between the algorithms is not very large.

In terms of memory usage, Apriori and FP-Growth require less memory than Eclat.

- The chess dataset is dense, meaning that transactions contain many items. Because of this, the number of frequent patterns grows very quickly when the minimum support decreases.

The results show that Apriori becomes very slow on this dataset. For example, when the support is 0.8, Apriori takes more than 700 seconds, while Eclat and FP-Growth take only a few seconds. When the support becomes even smaller, the execution time of Apriori increases dramatically. Eclat and FP-Growth perform much better on this dataset. However, they require much more memory.

- The accidents dataset is large but less dense than chess. On this dataset, Eclat achieves the best execution time. However, the memory usage of Eclat is very high. When the support decreases towards 0.7, Apriori becomes extremely slow and its execution time grows rapidly. FP-Growth provides a good compromise between execution time and memory usage.
- When the minimum support decreases, the number of frequent patterns increases. This makes the mining process more expensive and increases both execution time and memory usage.

### 4.1 Summary

Overall, the experiments show that the choice of the algorithm depends on the dataset.

Apriori uses little memory but becomes very slow when the dataset is dense.

Eclat is usually very fast but requires a large amount of memory.

FP-Growth often provides a good balance between execution time and memory usage.